

Design Patterns în C# — Teorie

Modul construit lecție cu lecție · corespondent exercițiilor din repo

Lecția 0 — Ce este un design pattern (puntea de la interfețe)

De unde venim

La lecția trecută ai învățat **interfețele**: o clasă semnează un contract (**can-do**), iar restul codului depinde de contract, nu de clasa concretă. **Desen** lucra cu `IElement []` fără să-i pese CE sunt elementele — doar CĂ respectă contractul.

Ține minte propoziția asta, pentru că e singura idee din tot modulul:

Depinzi de contract, nu de clasă.

Toate cele 8 pattern-uri pe care le facem sunt aceeași idee, aplicată la trei întrebări diferite.

Ce e, de fapt, un pattern

Un **design pattern** nu e o bibliotecă, o clasă gata făcută sau o sintaxă nouă. E un **nume pentru o soluție care se repetă** la o problemă care se repetă. Cineva a lovit aceeași problemă de o mie de ori, a găsit forma care merge, și i-a dat un nume ca să putem vorbi despre ea în două cuvinte în loc de zece minute.

Beneficiul e dublu:

- **Nu reinventezi** — când recunoști problema, știi deja forma soluției.
- **Comunici scurt** — spui „aici e un Strategy” și tot echipa înțelege structura, fără să citească fiecare linie.

Riscul, pe care îl vom evita: pattern-ul aplicat unde nu trebuie. Un pattern rezolvă o durere concretă. **Fără durere, fără pattern** — altfel adaugi complexitate degeaba.

Firul logic al modulului — trei întrebări

Grup	Întrebarea la care răspunde	Pattern-uri
Comportament	<i>cine se comportă și cum?</i>	Strategy, Observer, State
Structură	<i>cum compun obiectele?</i>	Decorator, Adapter
Creare	<i>cine și cum creează obiectele?</i>	Factory Method, Builder, Singleton

Ordinea nu e întâmplătoare: începem cu **comportamentul**, pentru că e cel mai aproape de interfețe. Primul pattern — Strategy — e practic un pas mic peste ce știi deja.

Lecția 1 — Strategy

Un cuvânt: comportament interschimbabil. Închizi fiecare variantă de „cum se face ceva” într-o clasă separată care semnează același contract, și o schimbi când vrei — chiar la runtime.

Problema (exercițiul ex1: costul de livrare)

Un magazin calculează costul de livrare al unei comenzi. Costul depinde de metodă: standard, express, promoție gratuită. Prima variantă la care se gândește oricine:

```
public decimal CalculeazaCost(TipLivrare tip, decimal greutate, decimal distanta)
{
    switch (tip)
    {
        case TipLivrare.Standard: return 5 + 0.5m * greutate + 0.1m * distanta;
        case TipLivrare.Express:  return 12 + 1.0m * greutate + 0.25m * distanta;
        case TipLivrare.Gratuita:  return 0;
        default: throw new ArgumentException("Unknown delivery type");
    }
}
```

Merge. Și e o capcană. Întreabă-te ce faci când firma adaugă `LivrareInternationala`:

- deschizi metoda asta și adaugi un `case`;
- probabil mai există un `switch` la fel în alt loc (afișare, facturare) — îl deschizi și pe ăla;
- fiecare `switch` uitat = un bug tăcut.

Ai lovit exact regula **open/closed** de la interfaces ex5: *o clasă ar trebui să fie deschisă la extindere, dar închisă la modificare*. Aici, ca să extinzi (metodă nouă), ești **obligat să modifice** cod care deja mergea. `switch`-ul pe tip e mirosul clasic care strigă „aici lipsește un pattern”.

Ideea

Întoarce lucrurile pe dos. În loc ca o metodă să știe TOATE variantele și să aleagă cu `switch`, **fiecare variantă devine o clasă** care știe DOAR de ea și semnează un contract comun:

```
public interface ILivrareStrategie
{
    string Nume { get; }
    decimal CalculeazaCost(decimal greutateKg, decimal distantaKm);
}
```

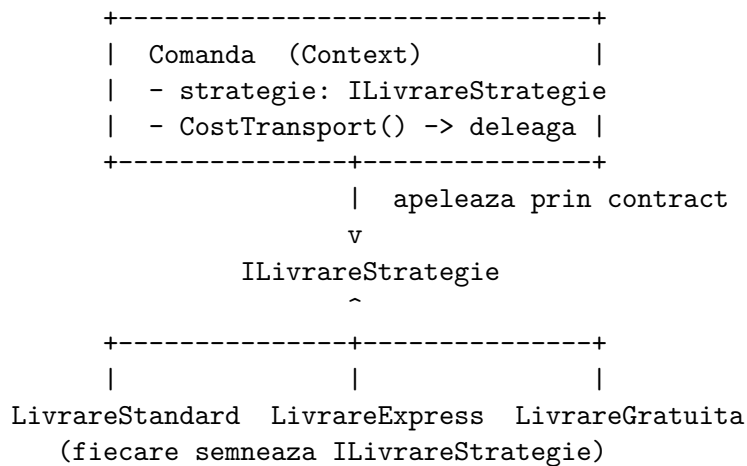
Comanda nu mai conține nicio formulă și niciun `switch`. Ține o `ILivrareStrategie` și o întreabă:

```
public decimal CostTransport()
{
    return strategie.CalculeazaCost(GreutateKg, DistanțaKm);
}
```

O metodă nouă de livrare = **o clasă nouă**, atât. Nu atingi Comanda, nu atingi celelalte strategii. Închis la modificare, deschis la extindere — de data asta chiar.

Cele trei roluri

Rol	În exercițiu	Ce face
Strategy (contractul)	<code>ILivrareStrategie</code>	declară operația interschimbabilă
ConcreteStrategy	<code>LivrareStandard</code> , <code>LivrareExpress</code> , <code>LivrareGratuita</code>	fiecare, o singură variantă a algoritmului
Context	<code>Comanda</code>	ține o strategie, delegă spre ea, o poate schimba



Mecanismul de dedesubt

Nu e magie — sunt trei lucruri pe care le știi deja, puse împreună:

1. **Polimorfism prin contract.** `strategie.CalculeazaCost(...)` cheamă implementarea clasei reale din spatele interfeței. Exact ce făcea `IElement.Afisare()` la desen.
2. **Delegare.** `Comanda` nu *face* calculul, îl *pasează* mai departe. Contextul e un dispecer, nu un executant.
3. **Injectare.** Strategia intră din afară (prin constructor sau `SchimbaStrategie`), nu e creată înăuntru. De-aia o poți schimba la runtime — și de-aia contextul nu depinde de nicio clasă concretă.

Detaliul care le leagă: `CalculeazaCost` primește greutatea și distanța ca **parametri**, nu le ține strategia în câmpuri. Așa aceeași instanță `LivrareStandard` servește orice comandă — strategia e *fără stare*, deci refolosibilă și fără surprize. (Vezi întrebarea din bonusul exercițiului.)

Când îl folosești

- Ai **mai multe variante** ale aceluiași lucru (calcul, sortare, validare, format de export) și vrei să comuți între ele.
- Vezi un **switch/lanț de if** pe un „tip” care se tot repetă prin cod.
- Vrei să **adaugi variante** fără să atingi codul existent.
- Vrei să poți schimba comportamentul **la runtime**, nu doar la compilare.

Când NU îl folosești

- Ai o **singură variantă** și nu se întrevăd altele. Un `if` simplu e mai onest decât trei clase și o interfață.
- Variantele diferă printr-o **singură valoare**, nu printr-un algoritm (atunci e un parametru, nu o strategie).
- Pattern-ul aplicat de dragul pattern-ului adaugă doar ceremonie. Amintește-ți: fără durere, fără pattern.

Capcane frecvente

- **Context cu switch rămas.** Dacă `Comanda` tot decide cu `if` care strategie să folosească, n-ai mutat problema — ai dublat-o. De-aia constrângerea din exercițiu interzice `if/switch` pe tip în `Comanda`.
- **Strategie cu stare de comandă.** Dacă `LivrareStandard` ține greutatea în câmp, n-o mai poți refolosi între comenzi. Ține strategia fără stare; datele vin ca parametri.
- **Prea multe strategii minuscule.** Dacă variantele diferă cu o linie, poate era destul un `Func<>` sau un parametru.

Legătura cu ce urmează

În bonusul exercițiului ai scris `LivrareCuReducere`: o strategie care primește **altă strategie** în constructor și îi ajustează rezultatul. O strategie care împachetează o strategie — asta e deja un **Decorator** (lecția 4). Strategy ți l-a arătat gratis; peste trei lecții doar îi punem numele.

Document viu — crește cu fiecare lecție. Următoarea: Lecția 2 — Observer.